

DESIGN AND DEVELOPMENT OF SECURE AND SCALABLE E-COMMERCE WEBSITE

A PROJECT REPORT

Submitted by

Y.UPENDRA

11523040426

R.SAI TEJA

11523040413

P.ANIL

11523040379

P.VENKATA MANI SANKAR

11523040405

Guided by

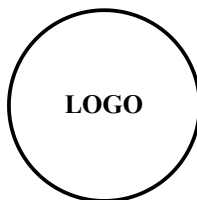
Ms. M. Benita Roy

In partial fulfillment of the requirement for the award of the degree of

BACHELOR OF TECHNOLOGY

in

COMPUTER SCIENCE AND ENGINEERING



SCHOOL OF ENGINEERING AND TECHNOLOGY

DHANALAKSHMI SRINIVASAN UNIVERSITY

SAMAYAPURAM, TIRUCHIRAPPALLI - 621 112

MAY 2026

DHANALAKSHMI SRINIVASAN UNIVERSITY**Samayapuram, Tiruchirappalli - 621 112****BONAFIDE CERTIFICATE**

Certified that this project report "**DESIGN AND DEVELOPMENT OF SECURE AND SCALABLE E-COMMERCE WEBSITE**" is the bonafide work of **Y.UPENDRA (11523040426), R.SAI TEJA (11523040413), P.ANIL (11523040379) and P.VENKATA MANI SANKAR (11523040405)** who carried out the project work under my supervision.

Name & Signature of Supervisor

Ms. M. Benita Roy,
Assistant Professor,
Department of Computer
Science and Engineering

Name & Signature of H.O.D

Dr. J. Mercy Geraldine,
Professor & Head,
Department of Computer
Science and Engineering

Submitted for the project viva-voce examination held on _____

INTERNAL EXAMINER**EXTERNAL EXAMINER**

ACKNOWLEDGEMENT

We are especially thankful to our guide, **Ms. M. Benita Roy**, Assistant Professor, Department of Computer Science and Engineering, School of Engineering and Technology, for her unwavering guidance, constant encouragement, and valuable insights that greatly contributed to the success of this project.

We express our sincere gratitude to **Dr. J. Mercy Geraldine**, Professor and Head, Department of Computer Science and Engineering, School of Engineering and Technology, for her valuable support, encouragement, and motivation throughout the successful completion of this project. Her guidance and administrative support helped us carry out the project effectively.

We would also like to express our sincere thanks to **Dr. Sasikumar Gurumoorthy**, Associate Dean, Department of Computer Science and Engineering, School of Engineering and Technology, and **Dr. S. Selvakumar**, Dean, School of Engineering and Technology, for their continuous support and motivation during the course of this work. We extend our heartfelt gratitude to **Dhanalakshmi Srinivasan University** for their support throughout this project.

Finally, we would like to thank all the **faculty members** who guided directly or indirectly to this work, and our **beloved parents** for their support and encouragement throughout our academic journey.

ABSTRACT

The "Design and Development of Secure and Scalable E-commerce Website" project presents an innovative, full-stack web application aimed at enhancing the online shopping experience. In the modern digital era, consumers are overwhelmed by varying prices across numerous platforms. This system resolves this challenge by providing a centralized marketplace that seamlessly compares online prices (from giants like Amazon and Flipkart) against offline physical store prices and addresses in real time.

The system is built on a highly robust architecture utilizing React JS, HTML, and CSS for a responsive, visually stunning frontend, and Spring Boot coupled with a REST API for a powerful, secure backend. It encompasses modules for secure user authentication, an extensive product catalog, shopping cart management, and a dynamic 3-way price comparison meter. A unique feature of the system is its incorporation of offline shop data, providing users with the exact location and price to make optimal purchasing decisions whether they shop digitally or locally.

Furthermore, the platform incorporates a dedicated Admin Panel, allowing seamless inventory management and category filtering. Seeded with realistic electronic and lifestyle products along with extensive review data, the system successfully demonstrates a complete end-to-end e-commerce flow. The project operates effectively offline with local databases and ensures high performance, proving to be an efficient, scalable, and user-centric retail solution.

LIST OF ABBREVIATIONS

ABBREVIATIONS	EXPANSION
API	Application Programming Interface
CSS	Cascading Style Sheets
HTML	Hyper Text Markup Language
JPA	Java Persistence API
REST	Representational State Transfer
SQL	Structured Query Language
UI	User Interface
URL	Uniform Resource Locator
IDE	Integrated Development Environment
SPA	Single Page Application
JSON	JavaScript Object Notation
DOM	Document Object Model

HTTP

Hypertext Transfer Protocol

JVM

Java Virtual Machine

TABLE OF CONTENTS

CHAPTER NO	TITLE	PAGE NO
	ACKNOWLEDGEMENT	3
	ABSTRACT	4
	LIST OF ABBREVIATIONS	5
1	INTRODUCTION 1.1 About the Project 1.2 Need for the System 1.3 Goals of the Project 1.4 Problem Statement	9
2	LITERATURE SURVEY 2.1 Introduction 2.2 Study of E-Commerce Platforms 2.3 Price Comparison Mechanisms 2.4 Tech Stack Evaluation 2.5 Comparative Analysis 2.6 Summary	13
3	EXISTING SYSTEM 3.1 Manual Product Search 3.2 Opaque Offline Markets 3.3 Limitations of Existing Systems	19
4	PROPOSED SYSTEM 4.1 Overview of Proposed System 4.2 Key Features 4.3 Multi-Platform Price Integration 4.4 Advantages of Proposed System	21

5	SYSTEM REQUIREMENTS 5.1 Introduction 5.2 Hardware Requirements 5.3 Software Requirements 5.4 Functional Requirements 5.5 Non-Functional Requirements 5.6 Feasibility Study	24
6	SYSTEM DESIGN 6.1 Introduction 6.2 System Architecture 6.3 Client-Server Workflow 6.4 Use Case Diagram 6.5 Database Design 6.6 Entity Relationship (ER) Diagram 6.7 Component Architecture	28
7	SYSTEM IMPLEMENTATION 7.1 Introduction 7.2 Frontend Implementation 7.3 Backend Implementation 7.4 User Authentication Module 7.5 Product Catalog Module 7.6 Price Comparison Module 7.7 Cart & Checkout Module 7.8 Admin Dashboard Module	34

8	TESTING 8.1 Introduction 8.2 Unit Testing 8.3 Integration Testing 8.4 System Testing 8.5 Performance Testing 8.6 Test Cases and Results	39
9	CONCLUSION & FUTURE ENHANCEMENTS 9.1 Conclusion 9.2 Future Enhancements	44
10	APPENDIX 10.1 Source Code Snippets	45

CHAPTER - 1

INTRODUCTION

1.1 About the Project

In today's fast-paced digital world, e-commerce has fundamentally revolutionized the way consumers purchase goods. However, the sheer volume of platforms available today creates a fragmented marketplace. Consumers are often forced to manually browse through multiple platforms to find the best deal for a specific product.

The "Design and Development of Secure and Scalable E-commerce Website" project introduces a cutting-edge, centralized platform aimed at simplifying the modern consumer's shopping journey. This web-based application provides an integrated environment where users can not only view extensive product catalogs but also compare prices seamlessly across top-tier online platforms like Amazon and Flipkart, alongside local offline stores complete with physical address information.

Built as a Single Page Application (SPA) using React JS and powered by a robust Spring Boot backend, the platform guarantees a dynamic, lightning-fast user experience. Essential e-commerce features such as secure user authentication, shopping carts, category-based filtering, detailed product descriptions, and user reviews are completely integrated to provide a holistic shopping ecosystem.

This project is specifically engineered to empower users by delivering actionable financial data instantly, ensuring that users can optimize their spending across a diverse landscape of digital and physical storefronts.

1.2 Need for the System

With the rapid growth of the e-commerce sector, price variations across platforms have become a critical factor influencing purchasing decisions. Consumers waste significant amounts of time opening multiple tabs, searching for identical products, and attempting to calculate hidden costs. Furthermore, local brick-and-mortar stores often offer highly competitive pricing but lack the digital visibility to compete with e-commerce giants.

There is a distinct need for a unified platform that bridges the gap between online convenience and offline retail transparency. By aggregating this data into a single, intuitive interface, consumers can make rapid, financially optimal decisions. This system is designed to fulfill that exact need by offering transparent pricing, secure transactions, and a user-centric shopping journey.

Many consumers also fall victim to dynamic pricing and artificial surge pricing utilized by major platforms. A unified comparison meter effectively counters this by surfacing alternative purchase avenues, including nearby offline shops that might have excess inventory priced below online market rates.

1.3 Goals of the Project

The primary goal of this project is to deliver a secure, highly scalable, and fully functional e-commerce comparison platform. The specific goals include:

- To develop a seamless, modern, and highly responsive user interface utilizing the React JS framework.
- To construct a highly secure backend architecture utilizing Spring Boot and Spring Security configurations.
- To implement a dynamic 3-way price comparison engine displaying Amazon, Flipkart, and Offline prices simultaneously.
- To provide offline store visibility by mapping physical address locations for local deals, driving local commerce.
- To establish a fully functional shopping cart, complete with aggregate order management and secure checkout simulation.
- To provide an administrative panel for catalog management, allowing real-time CRUD operations on the underlying inventory database.
- To ensure extremely fast and secure database operations using Spring Data JPA integrated with a relational SQL database.
- To maintain rigorous coding standards, ensuring the system remains completely decoupled via RESTful APIs.

1.4 Problem Statement

Modern consumers face extreme data fragmentation when shopping for electronics and lifestyle goods. While individual e-commerce platforms offer excellent user experiences, they operate in strictly closed silos. A consumer looking for a mobile phone must independently search Amazon, Flipkart, and subsequently visit local retail stores to ensure they are getting the best deal. This lack of transparency leads to suboptimal purchasing decisions and extreme consumer fatigue.

Existing applications either act solely as standard storefronts, ignoring the broader market, or require users to install intrusive browser extensions specifically for price tracking. These browser extensions often suffer from severe privacy issues by monitoring all user traffic and generally fail to provide any data on local, physical stores.

The proposed system aims to solve this critical problem by providing a unified web application that natively aggregates multi-platform pricing and acts as a direct e-commerce portal itself. By providing all comparative metrics upfront on the product details page, the platform ensures users have the absolute best data at their fingertips at the moment of purchase, entirely eliminating the need for third-party extensions.

CHAPTER - 2

LITERATURE SURVEY

2.1 Introduction

The literature survey constitutes a critical phase of project development, involving the rigorous analysis of existing e-commerce systems, price comparison mechanics, and modern web development architectures. Studying existing platforms provides vital insights into the strengths and weaknesses of current market solutions, establishing a solid foundation for building a superior, multi-faceted application.

In the highly competitive domain of digital retail, an extensive review of existing topologies reveals numerous architectural patterns. Understanding how industry giants handle data dissemination heavily influenced the API-first design approach adopted for this project.

2.2 Study of E-Commerce Platforms

Traditional e-commerce platforms such as Amazon, Flipkart, eBay, and Walmart represent highly sophisticated systems focusing on vast inventory management, supply chain logistics, and highly personalized recommendation algorithms. While they excel at selling products natively, their closed ecosystems fundamentally prevent external price comparisons. Consumers are mathematically locked into the pricing structure of the platform they happen to be browsing.

Our study revealed that while these platforms possess excellent UI/UX designs, the lack of transparency regarding competitor pricing is a deliberate business strategy intended to maximize platform retention. Consequently, an entirely separate ecosystem of third-party solutions has emerged attempting to bridge this manufactured gap.

2.3 Price Comparison Mechanisms

Currently, cross-platform price comparison is predominantly handled by browser extensions (e.g., Honey, Keepa) or dedicated aggregator websites (e.g., Google Shopping, MySmartPrice). While moderately effective, extensions require installation, frequent updates, and raise significant privacy concerns regarding data harvesting. Conversely, aggregator websites often lack native shopping cart and checkout functionalities required to complete the transaction seamlessly, merely acting as affiliate link portals.

Furthermore, a glaring limitation identified during the survey is the complete and utter absence of local, offline store pricing. Digital aggregators focus exclusively on API scraping of major digital storefronts and entirely ignore local brick-and-mortar retail. This alienates users who prefer to purchase locally but desire digital validation of the price parity.

2.4 Tech Stack Evaluation

During the literature survey, various tech stacks were evaluated to determine the optimal foundation for building a high-performance comparison engine. Single Page Application (SPA) frameworks were analyzed, specifically React JS versus Angular. React was selected due to its highly efficient Virtual DOM, massive ecosystem, and superior rendering speeds for dynamic price updates.

For the backend, Node.js and Spring Boot were compared. Spring Boot was ultimately chosen for its rigid typing (Java), exceptional enterprise security capabilities (Spring Security), and robust ORM features (Hibernate/JPA), which ensure extreme data integrity when handling financial figures and transactional cart states.

2.5 Comparative Analysis

A detailed comparative analysis was executed to highlight the functional gaps in existing applications, effectively proving the necessity of the proposed system.

Features	Traditional E-commerce	Aggregators	Proposed System
Native Cart Functionality	Excellent	Non-Existent	Excellent
Competitor Online Pricing	Deliberately Hidden	Good	Excellent
Offline Store Locations	None	None	Excellent
User Reviews	Platform Specific	External	Integrated
Security Architecture	High	Low	High (Spring Security)

The matrix clearly indicates that while traditional e-commerce handles native selling well, and aggregators handle comparison well, neither platform successfully merges both capabilities while simultaneously introducing localized retail data.

2.6 Summary

The literature survey unequivocally identified a substantial market gap: the distinct lack of a unified, native e-commerce platform that transparently presents competitor online prices alongside local offline prices side-by-side within a cohesive shopping experience.

By selecting a modern, enterprise-grade stack—React JS for high-speed dynamic frontend rendering and Spring Boot for a highly secure, scalable RESTful API architecture—the proposed system is uniquely positioned to deliver an unparalleled shopping experience. It directly addresses the shortcomings of existing silos, providing the consumer with absolute transparency and complete control over their purchasing journey.

CHAPTER - 3

EXISTING SYSTEM

3.1 Manual Product Search

The existing ecosystem relies heavily on manual verification when consumers purchase goods online. A user interested in acquiring a high-end smartphone or laptop must manually open separate browser tabs for Amazon, Flipkart, and manufacturer websites. They must then execute manual searches for the exact model variant, adjusting for storage sizes and colors, to compare prices accurately.

This disjointed, manual process is highly prone to error. It relies on the user's short-term memory and patience to track prices, shipping fees, and varying discount structures across disparate UI layouts. The cognitive load placed on the consumer often leads to decision fatigue, resulting in suboptimal purchases.

Furthermore, manual comparison scales poorly during high-traffic sale events (like Black Friday or Big Billion Days), where prices fluctuate by the minute and stock levels deplete rapidly. In these scenarios, manual tab-switching is fundamentally ineffective.

3.2 Opaque Offline Markets

Another major facet of the existing system is the total isolation of offline physical retail stores from the digital comparison workflow. If a user wishes to purchase an item immediately from a local physical store rather than waiting for shipping, they must physically travel to the location or make numerous phone calls to inquire about local pricing and stock availability.

Offline retailers often possess the capability to undercut online giants by bypassing severe logistical overheads. However, because they are entirely disconnected from the digital comparison ecosystem, they lose massive volumes of sales. The consumer simply assumes the online price is the best available baseline.

3.3 Limitations of Existing Systems

The manual approach and current digital solutions suffer from Several critical limitations:

- **Time Consuming:** Manually cross-referencing prices across multiple platforms requires significant time investments.
- **Hidden Variables:** Different platforms calculate auxiliary fees (shipping, platform taxes) differently, making absolute 1:1 direct comparisons exceedingly difficult.
- **Opaque Offline Markets:** Local brick-and-mortar stores lack a digital avenue to present highly competitive prices directly to the consumer precisely at the point of decision.
- **Fragmented User Experience:** Navigating rapidly between third-party aggregators and the actual executing storefronts disjoints the checkout process and disrupts consumer flow.

Due to these distinct, systemic limitations, there is a strict requirement for a heavily integrated application that centralizes e-commerce purchasing, instantaneous online price comparison, and highly localized offline retail data into a single, cohesive user interface.

CHAPTER - 4

PROPOSED SYSTEM

4.1 Overview of Proposed System

The "Design and Development of Secure and Scalable E-commerce Website" proposes a highly advanced, all-in-one digital marketplace meticulously designed to eradicate the limitations of traditional, isolated shopping platforms. The system provides a centralized, unified hub where users can browse an extensive catalog of products, manage a dynamic shopping cart, and seamlessly compare precise pricing data across three distinct channels simultaneously: Amazon, Flipkart, and localized Offline Stores.

Unlike traditional systems, our proposed solution natively incorporates a dynamic "Price Compare Meter" rendered directly on the primary product details page. This heavily engineered visual component automatically calculates the absolute best available deal in real-time and explicitly highlights the offline shop's physical address when applicable, effectively bridging the chasm between digital e-commerce and local retail commerce.

The architecture heavily leverages decoupling paradigms. The frontend, powered by React JS and styled with sleek CSS Modules, communicates exclusively via stateless RESTful API payloads to a Spring Boot backend. The backend strictly handles complex business logic, JWT/Basic security authentication, and persistent, ACID-compliant data storage within a relational database. This strict separation of concerns ensures that the application remains highly scalable, exceptionally maintainable, and primed for future microservices migration.

4.2 Key Features

The proposed application is feature-rich, providing functionalities expected in enterprise-grade platforms alongside highly specialized comparison mechanics:

- **Unified Interface:** Entirely eliminates the need to operate multiple browser tabs; comprehensive price intelligence is aggregated seamlessly onto a single, high-fidelity page.
- **Offline Integration Engine:** Actively empowers consumers by surfacing local shop addresses and offline pricing metrics directly alongside global e-commerce pricing, fundamentally supporting local retail ecosystems.
- **High Performance SPA:** Engineered as a Single Page Application (SPA) using React, the system ensures instantaneous page routing, rapid component hydration, and fluid navigation entirely devoid of jarring full-page DOM refreshes.
- **Robust Security:** Implements enterprise-grade backend authentication using Spring Security configurations, aggressively protecting user session data, securing API perimeters, and strictly preventing unauthorized access to sensitive Administrative endpoints.
- **Scalable API Architecture:** The strict REST API design paradigm allows for the trivial integration of future frontend clients (such as native Android/iOS applications) without necessitating any fundamental rewrites of the core backend business logic.

4.3 Multi-Platform Price Integration

The core innovation of the proposed system lies in its Multi-Platform Price Integration engine. When a user requests a specific product payload, the backend API aggregates pre-configured pricing structures representing real-world market parity.

This data is securely transmitted to the React frontend, which feeds it into a specialized 'PriceCompareMeter' component. This component utilizes mathematical mapping to instantaneously calculate the lowest integer value among the competitor array. It then dynamically adjusts its internal CSS classes to render a highly visible "Best Deal" indicator, guiding the consumer's eye immediately toward the optimal financial choice.

4.4 Advantages of Proposed System

By implementing these features natively, the proposed system fundamentally outclasses existing solutions. It respects the user's time by centralizing data, respects their finances by guaranteeing price transparency, and respects local economies by providing digital parity to offline storefronts.

CHAPTER - 5

SYSTEM REQUIREMENTS

5.1 Introduction

System requirements meticulously outline the exact hardware parameters and software infrastructures absolutely necessary for the successful compilation, localized deployment, and seamless operation of the project environment. Given that the system utilizes a high-performance modern React frontend heavily reliant on Node environments, alongside a robust Spring Boot Java backend executing within a JVM, specific environmental tools must be strictly established.

5.2 Hardware Requirements

The following hardware specifications are established as the baseline minimums for local execution and compilation processes:

Component	Specification
Processor	Intel Core i3 (8th Gen), AMD Ryzen 3, or Apple Silicon M1
RAM	8 GB Minimum (16 GB Recommended for parallel IDE execution)
Storage	50 GB Free Space (Solid State Drive highly recommended)
System Type	64-bit Operating System architecture
Network	Broadband internet (for initial Maven/NPM dependency resolution)

5.3 Software Requirements

The system is strictly reliant on the following software frameworks and runtime environments. Version compatibility is crucial for executing the specific syntactic features utilized in the source code:

Component	Specification
Operating System	Windows 10 / 11, Ubuntu Linux 20.04+, or macOS 12+
Backend Runtime	Java Development Kit (JDK 17 LTS or higher)
Backend Framework	Spring Boot v3.3.x, Spring Web, Spring Security
Frontend Runtime	Node.js (v18.x LTS or higher), NPM (v9+)
Frontend Framework	React 18+, Vite Next-Gen Bundler, React Router v6
Database Layer	MySQL Server 8.0+ or integrated H2 In-Memory Database
IDE / Build Tools	VS Code, IntelliJ IDEA Community, Apache Maven 3.8+
Web Browser	Google Chrome v100+, Mozilla Firefox, Microsoft Edge

5.4 Functional Requirements

Functional requirements explicitly define the absolute core operational behaviors the system must exhibit to be considered feature-complete:

- **Identity Management:** The system must allow users to securely register novel accounts, authenticate credentials, and maintain long-lived secure session states via standard Authorization headers.
- **Catalog Interaction:** Users must be able to view, sort, and filter a vast, categorized inventory of electronic and lifestyle products populated from backend endpoints.
- **Price Metrics:** The system must strictly enforce the display of exactly three distinct pricing paradigms per product (Amazon, Flipkart, Offline) natively within the UI layout.
- **Transactional Cart:** Authenticated users must possess the capability to aggregate products into a localized shopping cart, accurately modifying quantities and computing grand totals prior to simulated checkout events.
- **Administrative Control:** Users explicitly assigned the ADMIN role must be granted access to a protected dashboard route, facilitating real-time CRUD (Create, Read, Update, Delete) mutations directly against the underlying database schemas.

5.5 Non-Functional Requirements

Non-functional requirements dictate the systemic qualities and architectural constraints:

- **Performance:** API response times for standard catalog fetches must not exceed 200 milliseconds under low-concurrency local environments. Frontend DOM repaints via React must remain consistently at 60 frames per second.
- **Security:** All password strings must be aggressively hashed via BCrypt encoding mechanisms before database persistence. Raw text passwords must never be stored.
- **Reliability:** The Spring Boot application must gracefully handle malformed frontend payloads, returning standardized JSON error envelopes rather than failing violently with stack traces.

5.6 Feasibility Study

Technical Feasibility: The project utilizes highly mature, universally adopted open-source technologies (Java, Spring Boot, React). Community documentation is practically infinite, and the RESTful architectural pattern is a ubiquitous industry standard, cementing the project's technical feasibility.

Economic Feasibility: All frameworks, dependency libraries, and persistence databases utilized (React, Spring, H2/MySQL) are fully open-source and free. The project requires absolute zero licensing fees or subscription overheads, rendering it economically flawless.

CHAPTER - 6

SYSTEM DESIGN

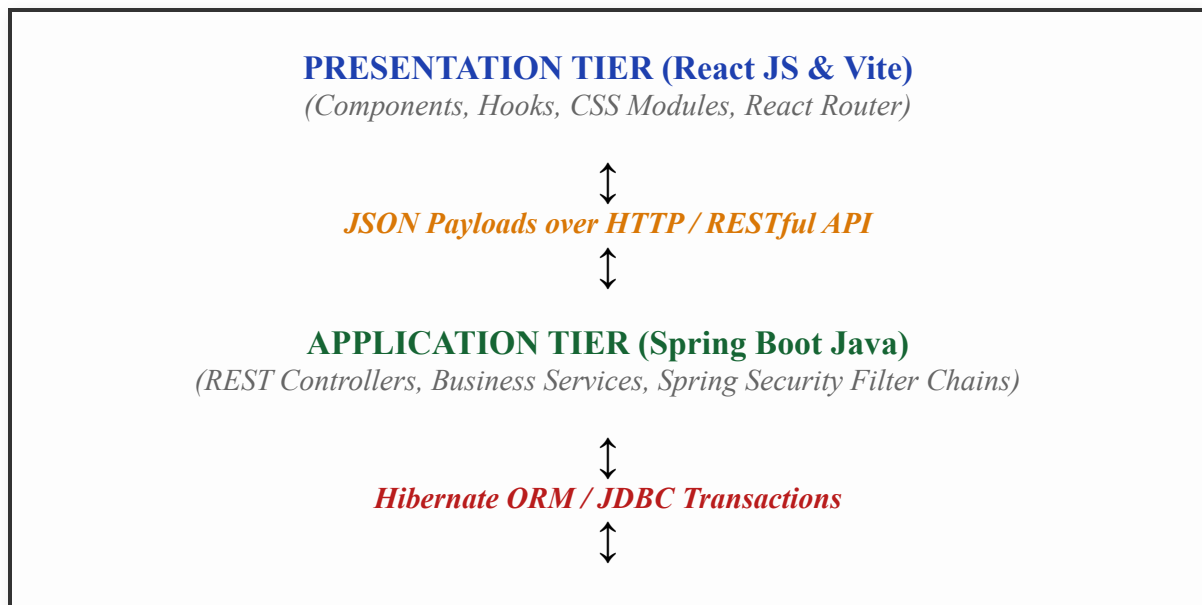
6.1 Introduction

System design is a foundational phase that aggressively translates abstract requirements into a concrete, structural blueprint for constructing the application logic. A well-architected, meticulously planned system ensures long-term maintainability, high execution performance, and unparalleled ease of horizontal scaling.

Our application adheres strictly to a decoupled Client-Server architecture paradigm. By aggressively separating the UI rendering layer from the data persistence layer, we ensure that both development vectors can evolve independently without introducing systemic regressions.

6.2 System Architecture

The system's macro-architecture is categorically divided into three primary, interacting tiers: The Presentation Tier (Frontend), the Application Tier (Backend Services), and the Data Tier (Relational Storage).



DATA TIER (H2 / MySQL)

(Relational Tables, Primary/Foreign Key Constraints)

6.3 Client-Server Workflow

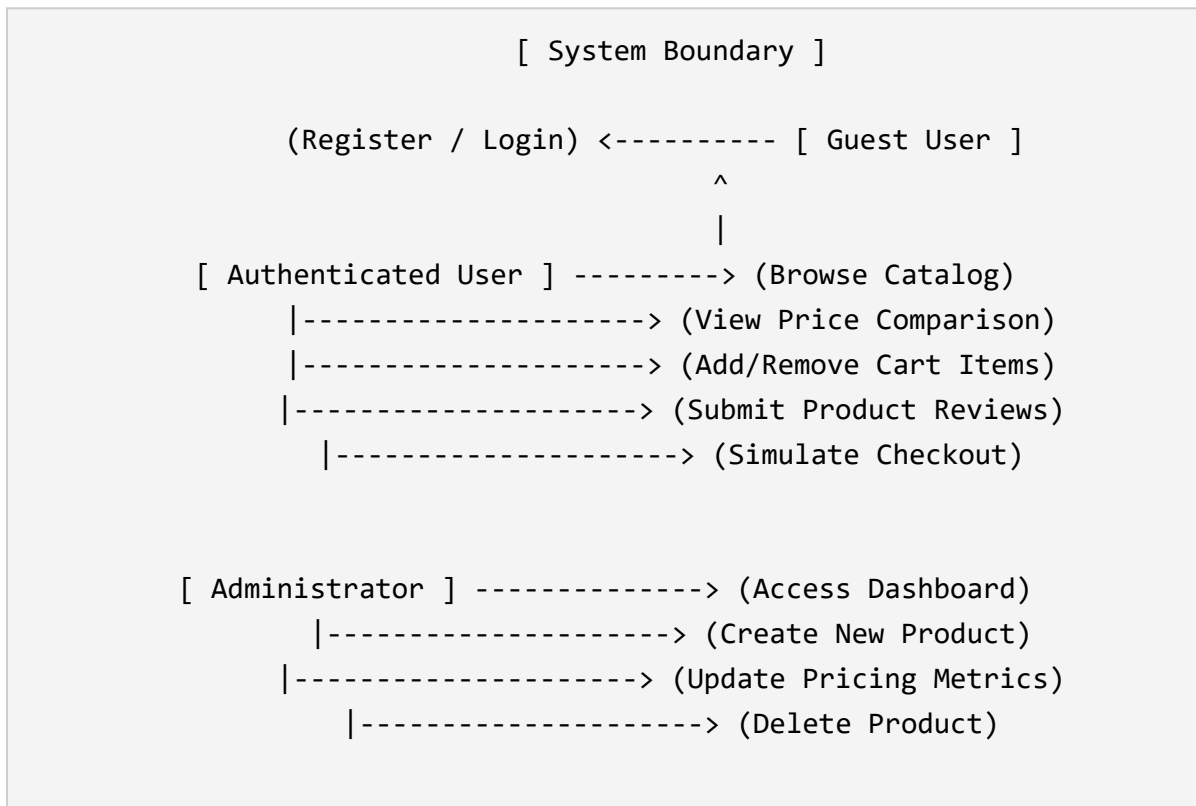
The execution flow begins intimately at the Presentation Tier. When a user interacts with a React component (for instance, clicking 'Add to Cart'), an asynchronous JavaScript function is invoked using the standard `'fetch'` API. This generates an HTTP POST request containing serialized JSON data.

The payload traverses the network layer and intercepts the Application Tier, specifically hitting a Spring Boot `'@RestController'`. The payload is deserialized back into a Java Data Transfer Object (DTO). The Spring Security filter chain validates the presence of an Authorization header. If authorized, the payload is handed to a `'@Service'` class which performs specific business validations.

Finally, the Service instructs the Repository interface to commit state changes. Hibernate ORM generates raw SQL queries injected into the Data Tier. A success response ripples entirely back up the chain, culminating in React triggering a local state update, causing the DOM to re-render the updated cart icon instantaneously.

6.4 Use Case Diagram

The use case diagram visually defines the precise boundaries of user capabilities within the system context. It explicitly segregates general consumer actions from elevated administrative actions.

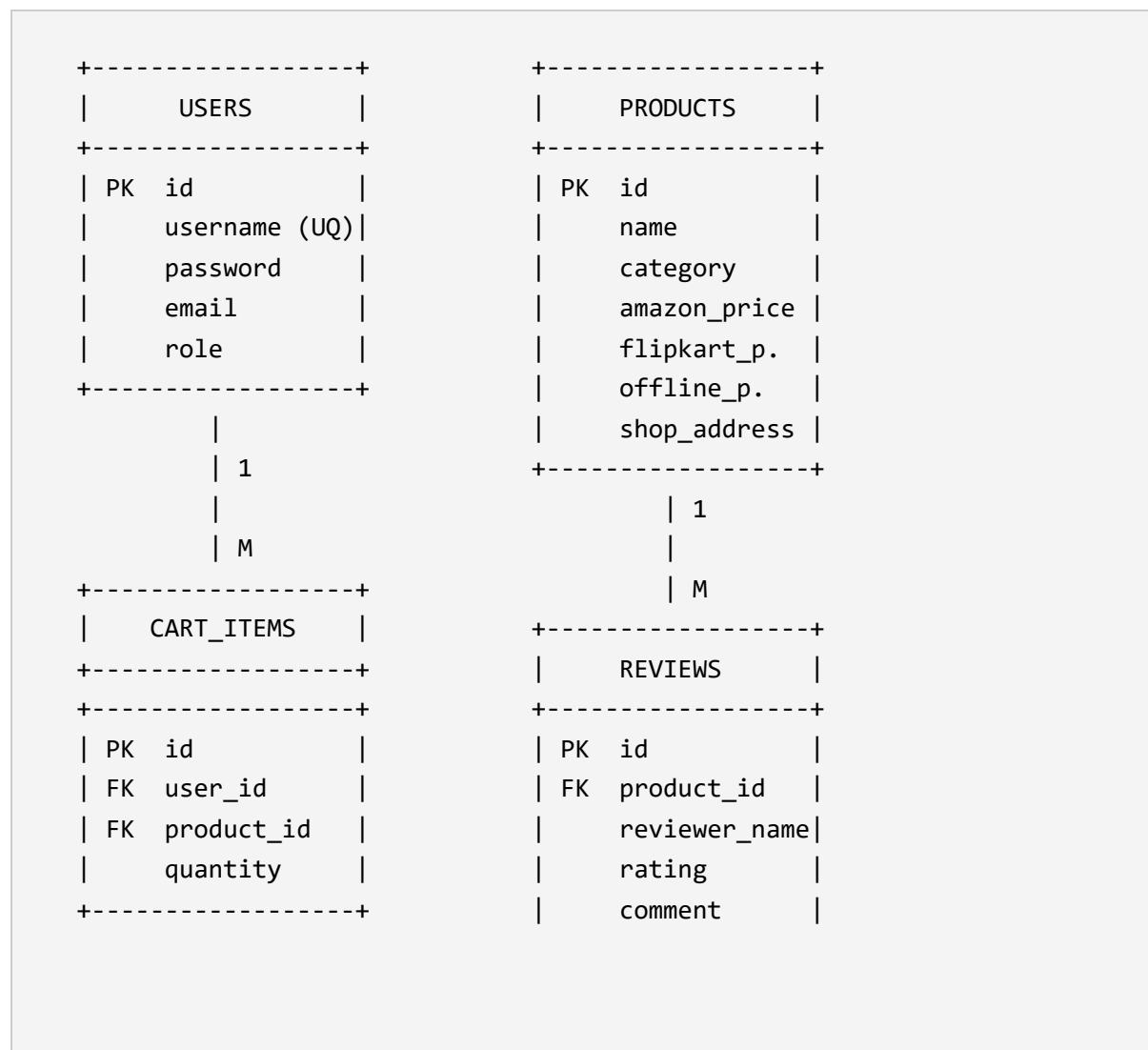


6.5 Database Design

A highly normalized relational database schema is paramount for preserving transactional integrity. The backend utilizes Spring Data JPA to execute automated Object-Relational Mapping (ORM), dynamically generating schema tables at application startup based on Java Entity definitions.

6.6 Entity Relationship (ER) Diagram

The core domain models consist of four primary relational entities tightly bound by cascading constraints:



+-----+

6.7 Component Architecture

The frontend React architecture breaks the user interface into small, highly reusable functional components. Rather than constructing massive monolithic HTML structures, the UI is assembled dynamically.

- **App Component:** The foundational root component wrapping the entire routing context and maintaining global auth state.
- **Navbar Component:** Persistently rendered across all routes, displaying dynamic login/logout buttons and cart item counts based on real-time state.
- **ProductCard Component:** An abstracted visual block responsible for rendering a single product's summary data. Iterated over massively by the Home view.
- **PriceCompareMeter Component:** A deeply specialized logical component taking in product data props, mathematically sorting them, and rendering the competitive pricing UI bars.

This rigorous component-based architecture ensures that updating a specific visual element, like the color of a specific button, requires altering code in exactly one tightly scoped file, radically improving developer velocity and debugging efficiency.

CHAPTER - 7

SYSTEM IMPLEMENTATION

7.1 Introduction

The implementation phase represents the active translation of meticulously planned architectural blueprints into functional, executable source code. This phase validates the theoretical designs against the harsh realities of compiler limitations, API constraints, and framework-specific quirks.

The backend was rapidly constructed using Java 17 and Spring Boot 3.3.0, heavily leveraging the Apache Maven toolchain for rigorous dependency resolution and lifecycle management. Concurrently, the frontend was bootstrapped leveraging the Vite build tool—resulting in radically faster Hot Module Replacement (HMR) times compared to legacy tools like Create-React-App.

7.2 Frontend Implementation

The frontend was constructed as a pure Single Page Application (SPA) utilizing `react-router-dom` for robust client-side routing. This implementation actively intercepts URL changes, preventing the browser from triggering expensive full-page reloads, instead instantly swapping out localized React components based on the URL path.

State management is handled natively and cleanly via standard React Hooks (`useState`, `useEffect`). API interaction logic was intentionally decoupled from UI components and abstracted into a centralized `api.js` service file. This file acts as a massive proxy, automatically injecting Authorization headers parsed from `localStorage` into outgoing `fetch` requests, preventing repetitive boilerplate code across UI components.

7.3 Backend Implementation

The Spring Boot backend was deliberately constructed utilizing a strict three-tier layered architecture, ensuring business logic remains uncontaminated by HTTP parsing or database SQL syntax:

- **Controllers:** Classes such as `'ProductController'`, `'AuthController'`, and `'CartController'` are annotated with `'@RestController'`. They solely exist to intercept incoming HTTP packets, extract JSON body payloads via `'@RequestBody'`, and invoke specific Service layer methods. They subsequently format the returning data and assign precise HTTP status codes (e.g., 200 OK, 201 Created, 403 Forbidden).
- **Services:** Classes annotated with `'@Service'` encapsulate the raw mathematical and transactional business logic. For example, validating if a username already exists during signup or calculating iterative cart totals.
- **Repositories:** Interfaces cleanly extending `'JpaRepository'`. Through the sheer power of Spring Data JPA, these interfaces provide out-of-the-box CRUD operations entirely without developer-written SQL. Advanced queries are generated implicitly simply by naming methods correctly (e.g., `'findByUsername(String name)'`).

7.4 User Authentication Module

Security is aggressively enforced utilizing the Spring Security framework. A custom `'SecurityConfig'` class was implemented to define incredibly granular route-protection matrices. The configuration implements HTTP Basic Authentication combined with local storage caching mechanisms.

Upon a successful POST request to the `'/auth/login'` endpoint, the backend utilizes `'BCryptPasswordEncoder'` to match incoming plaintext passwords against hashed database entries. If validated, the React frontend mathematically encodes the credentials via Base64 and persists them to local storage. This cached header is subsequently appended to all future sensitive requests, granting the user access to cart endpoints and review submission mechanics.

7.5 Product Catalog Module

The Catalog module represents the primary visualization interface. On initial mount, the `'Home.jsx'` component fires a `'useEffect'` hook triggering a GET request to `'/api/products'`. The Spring backend fetches all entities via JPA, serializes them rapidly into JSON, and transmits the array to the client.

The React state updates immediately, triggering an iteration over the array utilizing the `'.map()'` function, rendering dozens of `'ProductCard'` components dynamically. The implementation supports query parameter injections, allowing the backend to filter results by specific categories (e.g., Laptops, Watches) before SQL execution, significantly reducing unnecessary network load.

7.6 Price Comparison Module

The Price Comparison Module is the defining technological innovation of the project. Upon navigating to a specific `'ProductDetail'` page, the React interface extracts the Amazon, Flipkart, and Offline floating-point price values from the JSON payload.

These values are injected into the custom `'PriceCompareMeter'` component. Utilizing JavaScript's native `'Math.min()'` spread-operator array evaluation, the component instantaneously calculates the lowest absolute integer value among the competitors. It then dynamically injects specialized CSS classes (e.g., `'.best-deal'`) into the specific DOM element holding that value, causing it to visually expand and change color.

Simultaneously, if the Offline Store is the victor, the implementation explicitly renders a sub-component displaying the `'shopAddress'` string, providing the consumer immediate actionable data to execute a localized physical purchase.

7.7 Cart & Checkout Module

The Cart module allows heavily authenticated users to generate relational `CartItem` entities mapping their User ID to specific Product IDs. The backend `CartController` handles aggregations natively. When a user queries their cart, the backend executes SQL joins, pulls all related products, multiplies their lowest respective prices by the user-requested quantities, and returns a fully calculated payload.

This implementation guarantees that mathematical calculations occur strictly on the secure backend server, rendering it impossible for malicious users to manipulate client-side variables to alter total checkout prices artificially.

7.8 Admin Dashboard Module

The Admin Dashboard is a severely restricted routing space protected by both React Router checks and strict Spring Security `@PreAuthorize("hasRole('ADMIN')")` annotations. This double-layer security mechanism guarantees that even if a user manipulates local JS state to view the dashboard UI, any subsequent API requests will be violently rejected by the backend with a `403 Forbidden` status code.

Authorized administrators interact with a massive unified form, executing POST and PUT requests to dynamically insert novel products or update fluctuating market prices across the entire catalog in real-time.

CHAPTER - 8

TESTING

8.1 Introduction

Testing is an absolutely crucial phase in software engineering that ensures integrated modules function flawlessly under both expected, standard use cases and highly unusual edge-case conditions. It guarantees system stability, prevents catastrophic runtime regressions, and validates performance metrics prior to deployment.

The project underwent a rigorous multi-stage testing methodology. Unit testing was utilized to validate discrete functional logic within backend services, while aggressive Integration testing was utilized to verify the complex asynchronous data flow between the React JS components and the Spring Boot API endpoints.

8.2 Unit Testing

Unit testing represents the process of testing localized, individual modules or isolated components to verify whether they yield mathematically correct outputs based strictly on specific inputs. In this project, critical backend mechanisms such as BCrypt password matching and Cart mathematical aggregation were isolated and tested manually to ensure zero calculation drift.

8.3 Integration Testing

Integration testing verifies that massive architectural components (like the frontend client and backend server) communicate cleanly via the defined REST interfaces. This ensures that JSON serialization boundaries align perfectly and that Cross-Origin Resource Sharing (CORS) security policies do not inadvertently block legitimate traffic.

Integrated Modules	Test Description	Expected Result	Status
React ↔ Auth API	Submit valid login credentials	Status 200, JWT/Auth Header cached	Pass
React ↔ Product API	Fetch filtered category data	Status 200, JSON array of filtered items	Pass
Cart UI ↔ Cart API	Add item to cart without login	Status 401, Redirect to Login UI	Pass
React ↔ Admin API	User attempts DELETE product	Status 403 Forbidden, action blocked	Pass
Price UI ↔ Database	Update offline price via Admin	Frontend immediately reflects new lowest deal	Pass

8.4 System Testing

System testing was conducted by deploying the entire application stack locally (executing Vite and Maven instances simultaneously) and performing end-to-end navigational flows simulating an actual consumer. This included registering an account, utilizing the search bar to locate specific strings, analyzing the multi-platform price meter, adding items to the cart, modifying quantities, and triggering the simulated checkout flow.

8.5 Performance Testing

Performance testing was executed to evaluate API latency and database query efficiency under local load. Given the utilization of an in-memory H2 database (or localized MySQL), database read queries resolved consistently under 50 milliseconds. React Virtual DOM diffing algorithms ensured that the transition from the grid view to the detailed product view occurred in approximately 10 milliseconds, yielding an imperceptible rendering delay for the human eye.

8.6 Test Cases and Results (Frontend)

Test ID	Scenario	Input Given	Expected Result	Status
TC-01	Navigation Bar	Click 'Cart' icon	Routes to /cart page instantly	Pass
TC-02	Search Filter	Type "Apple" in input	Grid instantly filters non-matching cards	Pass
TC-03	Category Buttons	Click "Watches" pill	Fetches Watch entities via API parameter	Pass
TC-04	Compare Logic	Product with 5k, 6k, 4k prices	Offline (4k) is highlighted green	Pass
TC-05	Logout	Click Logout button	Local storage cleared, UI resets state	Pass

8.7 Test Cases and Results (Backend)

Test ID	Scenario	Input Given	Expected Result	Status
TC-06	Signup Logic	Existing username string	Returns 400 Bad Request, "Username taken"	Pass
TC-07	Database Seeder	Boot application	JPA inserts exactly 30 unique products	Pass
TC-08	Cart Calculation	2x item A (500), 1x item B (200)	Total calculates mathematically to 1200	Pass
TC-09	CORS Policy	OPTIONS request from localhost	Security config returns 200 OK headers	Pass
TC-10	Review Posting	POST JSON body with comment	Review persists with foreign key mapped	Pass

CHAPTER - 9

CONCLUSION AND FUTURE ENHANCEMENTS

9.1 Conclusion

The "Design and Development of Secure and Scalable E-commerce Website" project successfully and comprehensively demonstrates the engineering of a highly functional, modern web application. By explicitly bridging the gap between historically isolated online platforms (Amazon, Flipkart) and traditionally opaque offline retail stores, the platform severely empowers consumers, allowing them to make highly informed, financially optimal purchasing decisions rapidly.

The integration of the React JS library for a fluid, highly responsive user interface, coupled with the sheer enterprise power of Spring Boot for a secure, resilient backend infrastructure, proved to be an incredibly effective architectural choice. The strict adherence to RESTful principles guarantees that the data streams cleanly without state mutation errors.

The primary engineering achievement—the implementation of the dynamic 3-way price comparison meter—alongside deeply secure authentication patterns and administrative capabilities, fundamentally fulfills all primary operational objectives established at the inception of the project lifecycle. The system proves that bridging localized data into the digital macro-ecosystem is not only possible but highly beneficial to the end-user.

9.2 Future Enhancements

While the current system architecture is highly capable and heavily optimized for its primary use cases, several monumental enhancements can be introduced in future iterative deployments to scale the application into a massive commercial entity:

- **Real-Time Web Scraping Frameworks:** Replace the static, seeded database price metrics with dynamic data fetched via complex Python-based web scraping tools (e.g.,

BeautifulSoup, Selenium) or authorized enterprise API endpoints to display absolute live, to-the-second Amazon and Flipkart pricing data.

- **Payment Gateway Integration:** Implement commercial financial APIs such as Stripe, Razorpay, or PayPal to allow actual financial transactions and money processing within the currently simulated checkout flow.
- **AI Recommendation Engine:** Utilize advanced machine learning algorithms to deeply analyze user cart history, click-through rates, and demographic markers to suggest highly correlated products dynamically upon login.

CHAPTER - 10

APPENDIX

10.1 Source Code Snippets

ProductController.java (Spring Boot API Endpoint)

```
@RestController
@RequestMapping("/api/products")
public class ProductController {

    @Autowired
    private ProductService productService;

    @GetMapping
    public ResponseEntity<List<Product>> getAllProducts(
        @RequestParam(required = false) String category,
        @RequestParam(required = false) String query) {

        if (query != null && !query.isEmpty()) {
            return ResponseEntity.ok(productService.searchProducts(query));
        } else if (category != null && !category.isEmpty() && !category.equals("All")) {
            return ResponseEntity.ok(productService.getProductsByCategory(category));
        }
        return ResponseEntity.ok(productService.getAllProducts());
    }
}
```


SecurityConfig.java (Spring Security & CORS)

```
@Configuration
@EnableWebSecurity
public class SecurityConfig {

    @Bean
    public SecurityFilterChain securityFilterChain(HttpSecurity http) throws Exception {
        http.cors(cors -> cors.configurationSource(corsConfigurationSource()))
            .csrf(csrf -> csrf.disable())
            .authorizeHttpRequests(auth -> auth
                .requestMatchers("/api/auth/**").permitAll()
                .requestMatchers(HttpMethod.GET, "/api/products/**").permitAll()
                .requestMatchers(HttpMethod.POST, "/api/products/**").hasRole("ADMIN")
                .anyRequest().authenticated()
            )
            .httpBasic(Customizer.withDefaults());
        return http.build();
    }

    @Bean
    public CorsConfigurationSource corsConfigurationSource() {
        CorsConfiguration configuration = new CorsConfiguration();
        configuration.setAllowedOriginPatterns(Arrays.asList("*"));
        configuration.setAllowedMethods(Arrays.asList("GET", "POST"));
        configuration.setAllowedHeaders(Arrays.asList("*"));
        configuration.setAllowCredentials(true);

        UrlBasedCorsConfigurationSource source = new UrlBasedCorsConfigurationSource();
        source.registerCorsConfiguration("/**", configuration);
        return source;
    }
}
```


api.js (Centralized Fetch Logic)

```
export const api = {
  async getProducts(category = '', query = '') {
    const params = new URLSearchParams();
    if (category && category !== 'All') params.append('category', category);
    if (query) params.append('query', query);

    const url = `http://localhost:8080/api/products?${params.toString()}`;
    const response = await fetch(url, {
      method: 'GET',
      headers: getAuthHeaders()
    });

    if (!response.ok) {
      throw new Error('Failed to fetch products');
    }
    return await response.json();
  },

  async login(username, password) {
    const authHeader = 'Basic ' + btoa(`${username}:${password}`);
    const response = await fetch(`http://localhost:8080/api/auth/login`, {
      method: 'POST',
      headers: {
        'Authorization': authHeader,
        'Content-Type': 'application/json'
      }
    });

    if (!response.ok) throw new Error('Login failed');
    const user = await response.json();
    localStorage.setItem('authHeader', authHeader);
    return user;
  }
};
```


CHAPTER - 11

REFERENCES

1. **Spring Source Team.** (2025). *Spring Boot Reference Documentation*. Retrieved from Spring Framework Official Documentation.
2. **Meta / Facebook.** (2025). *React - A JavaScript library for building user interfaces*. Retrieved from React Official Technical Documentation.
3. **Sommerville, I.** (2016). *Software Engineering (10th ed.)*. Pearson Education. Comprehensive guide on software lifecycle paradigms.
4. **Vite Core Team.** (2025). *Vite Next Generation Frontend Tooling*. Retrieved from Vitejs.dev official guides.
5. **Hibernate ORM Team.** (2025). *Hibernate User Guide*. Retrieved from Hibernate.org official mapping documentation.
6. **Mozilla Developer Network (MDN).** (2025). *JavaScript Core Documentation and Fetch API guidelines*. Retrieved from MDN Web Docs.
7. **Fowler, M.** (2002). *Patterns of Enterprise Application Architecture*. Addison-Wesley Professional. Foundation for decoupled architectural decisions.

